Subject outline

Lecture topics (by week)

1 – Introduction to quantum computing and maths basics
2 – Single qubit representations and logic operations
3 – Two qubit states and logic gates
4 – Multi-qubit states and quantum arithmetic
5 – Basic quantum algorithms
6 – Period finding, cryptography and quantum factoring
7 – Shor's algorithm, post-quantum crypto, quantum key distribution
8 – Quantum search algorithms
9 – Grover search applications, optimisation problems
10 – Solving optimisation problems on quantum computers
11 – Applications in quantum machine learning
12 – Real quantum computer devices

Assignment schedule:
#1: Hand out in Week 2
#2: Hand out in Week 8

Week 5

Lecture 9

9.1 Algorithms
9.2 Example Classical Algorithms and Complexity Analysis
9.3 Quantum Algorithms

Lecture 10
10.1 Deutsch-Josza Algorithm
10.2 Simon's Algorithm

Practice class 5

Simple Quantum Algorithms

Recap: Lecture 9

Classical Algorithms

- Algorithms are set of instructions which when executed on a computer solves a computational problem.
- Examples:
- 1) Binary Search
- 2) Insertion Sort
- 3) Greatest Common Divisor (GCD)

Quantum Complexity

The total time to run an algorithm on a computer is: $T_n = C_e * f(n)$

Classical Computers :improves C_e

Quantum Computers :improves f(n)

Complexity Classes

- Algorithms are classified by asymptotic cost analysis
- For algorithm cost function f(n),
- f(n) = O(g(n)) if g(n) is upper bound to f(n)
- f(n) = Ω(g(n)) if g(n) is lower bound to f(n)
- The total time to run an algorithm on a
- Here "n" is the size of the input.

Intractable problems:

Problems for which no polynomial time algorithm exits

Example:

Sampling of a Random Quantum Circuit

10.1 Deutsch-Jozsa Algorithm

MULT20015

Lecture 10

Example 1: Deutsch-Jozsa algorithm

Problem: Given a Boolean function, f(x), determine if:

- f(x) is constant (always gives the same result), or
- f(x) is balanced (gives equal numbers of 0s and 1s)

Note:

Single qubit version -> Deutsch Algorithm

Multi qubit version -> Deutsch-Josza Algorithm

Applications:

- No well-defined real-world application
- First algorithm that showed quantum solution has fewer steps than any known classical solution
- Inspiration for Simon's algorithm which was then inspiration for Shor's famous algorithm

Classical Complexity

For an input "x" of "n" bits, the function f(x) is:

n=1

CONSTANT	f(0) = 0 f(1) = 0
	f(0) = 1 f(1) = 1
BALANCED	f(0) = 0 f(1) = 1
	f(0) = 1 f(1) = 0

n=2

CONSTANT	f(00) = 0 f(01) = 0 f(10) = 0 f(11) = 0
	f(00) = 1 f(01) = 1 f(10) = 1 f(11) = 1

BALANCED	f(00) = 0 f(01) = 0 f(10) = 1 f(11) = 1
	f(00) = 1 f(01) = 1 f(10) = 0 f(11) = 0

Elementary Operation: Evaluation of f(x) for n

Classical algorithm *(worst case) needs* $(2^n/2)+1$ *queries*

Cost = $(2^n/2)+1 = O(2^n)$

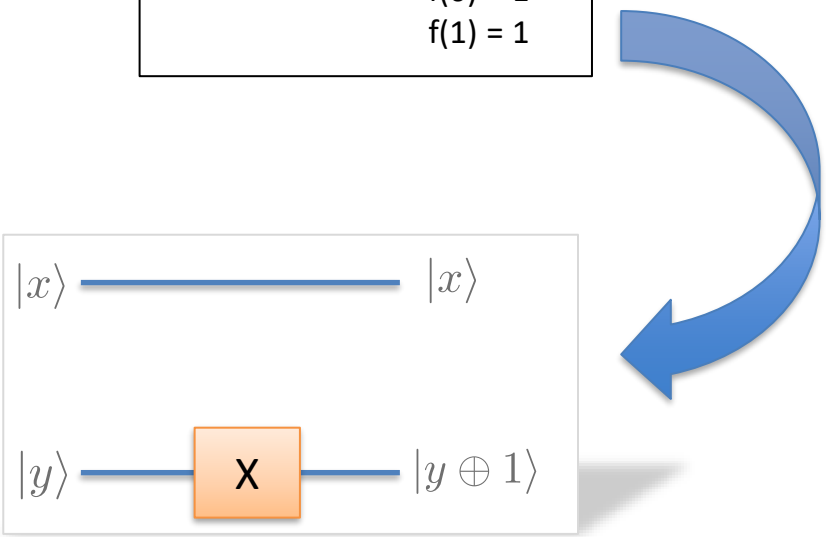
Quantum algorithm *(Deutsch-Josza) needs just 1 query*

Cost = 1 = O(1)

Example of a constant function

Recall for n=1:

CONSTANT	f(0) = 0 f(1) = 0
	f(0) = 1 f(1) = 1

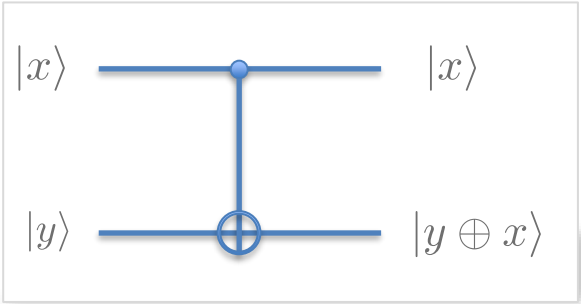


The circuit always flips (ie. NOT) the second qubit, regardless of the input.
This function is **constant**, since the output is always 1.

Example of a balanced function

Recall for n=1:

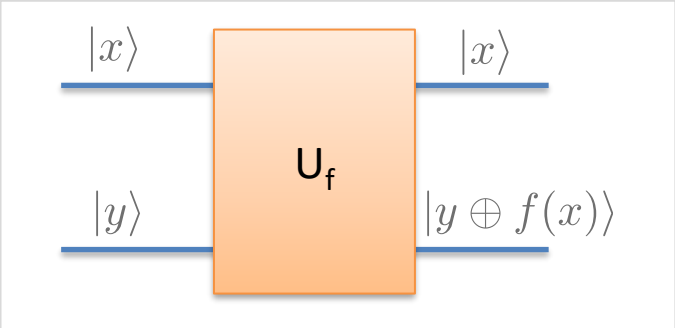
BALANCED	f(0) = 0 f(1) = 1
	f(0) = 1 f(1) = 0



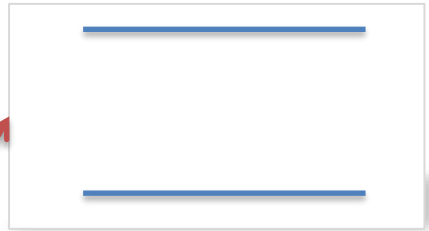
The circuit only flips (ie. NOT) the second qubit, if the input is a 1.
This function is **balanced**, since the output has equal numbers of 0 and 1 output.

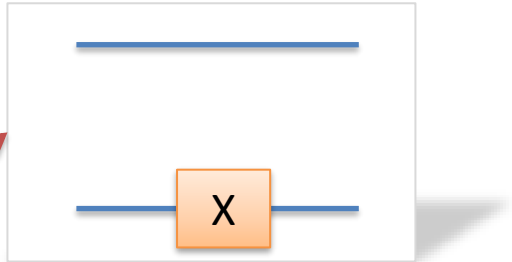
Deutsch-Jozsa algorithm

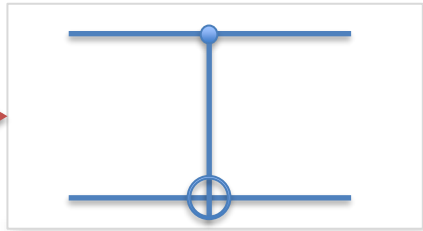
Let's take the example with just one bit/qubit input. There are 4 choices of function:

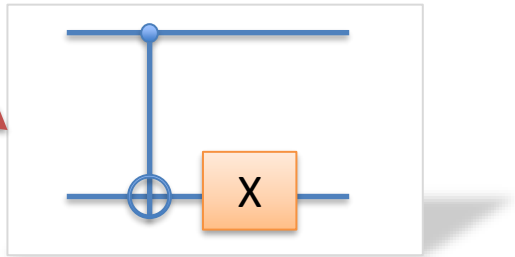


CONSTANT	$f_1(0) = 0$ $f_1(1) = 0$
	$f_2(0) = 1$ $f_2(1) = 1$
BALANCED	$f_3(0) = 0$ $f_3(1) = 1$
	$f_4(0) = 1$ $f_4(1) = 0$



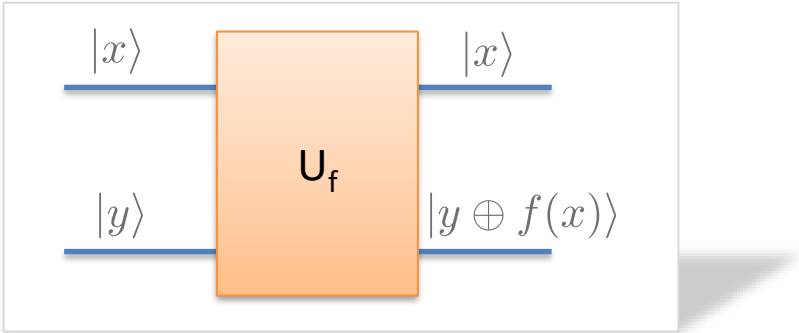






Deutsch-Jozsa algorithm

The general quantum circuit for Deutsch/Deutsch-Josza algorithm is:



Here U_f will be designed for constant/balanced implementations

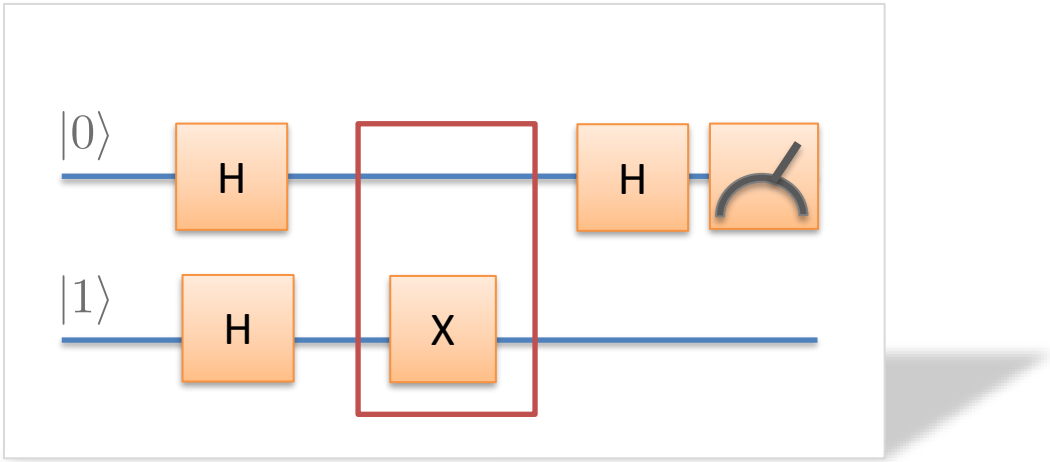
Recall from multi-qubit Lecture 8:

$|y\rangle = |0\rangle \rightarrow |y \oplus f(x)\rangle = |f(x)\rangle$

Therefore, $U_f|x\rangle|0\rangle = |x\rangle|f(x)\rangle$

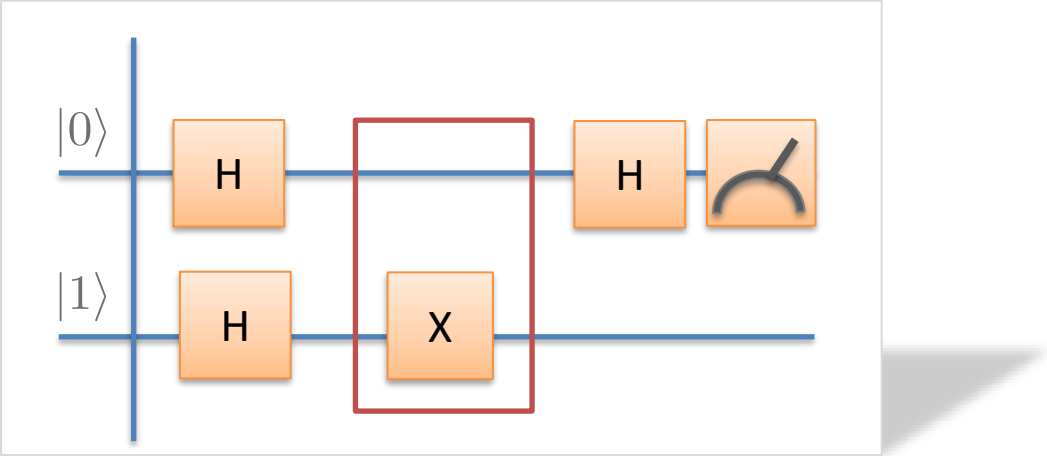
Deutsch algorithm(single bit functions): constant function

Let's carefully look at the working of quantum circuit for the constant case:



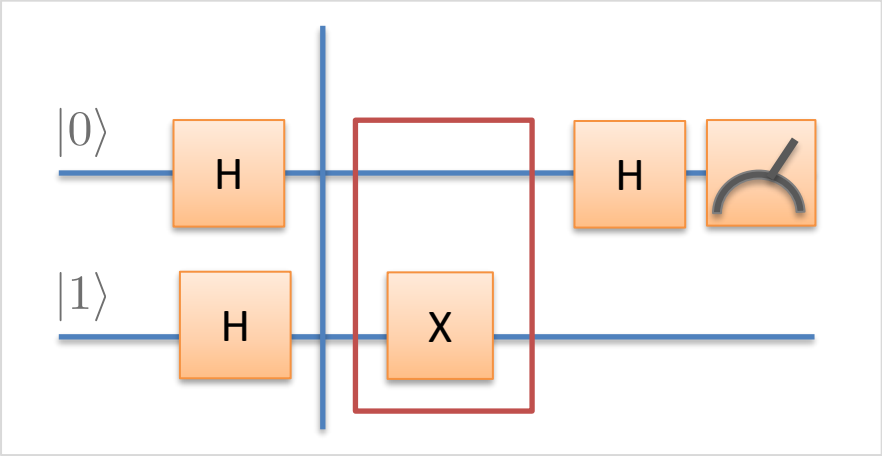
The function, U, is implemented by the gates inside this box

Deutsch algorithm: walkthrough



$$|\psi\rangle = |0\rangle \otimes |1\rangle$$

Deutsch algorithm: walkthrough



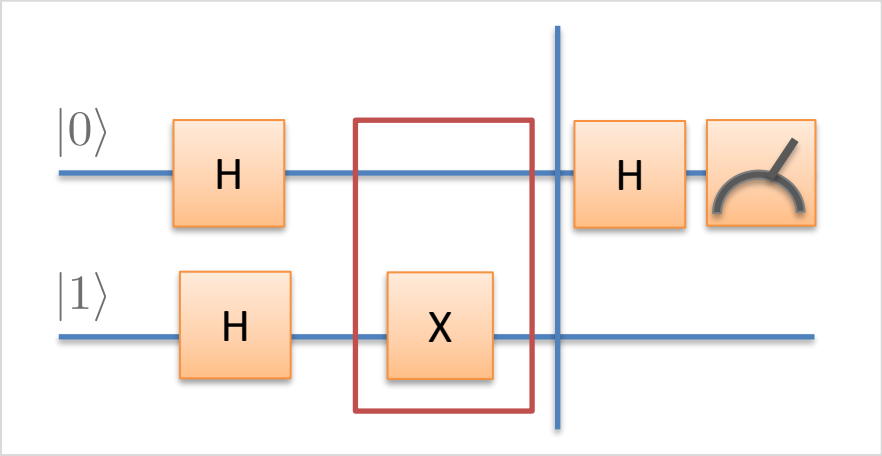
$$
\begin{aligned}
|\psi\rangle &= H\,|0\rangle \otimes H\,|1\rangle \\
&= \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
&= |+\rangle \otimes |-\rangle
\end{aligned}
$$

Note:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

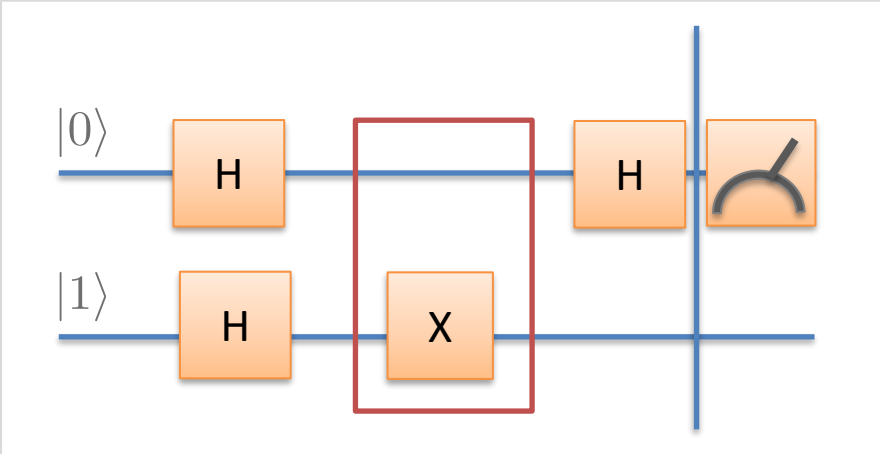
Deutsch algorithm: walkthrough



$$
\begin{aligned}
|\psi\rangle &= |+\rangle \otimes X\,|-\rangle \\
&= \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes X \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
&= \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes \frac{|1\rangle - |0\rangle}{\sqrt{2}} \\
&= -\,|+\rangle \otimes |-\rangle
\end{aligned}
$$

Global phase is unmeasurable

Deutsch algorithm: walkthrough



$$|\psi\rangle = -H\,|+\rangle \otimes |-\rangle$$
$$= -\,|0\rangle \otimes |-\rangle$$

Note:

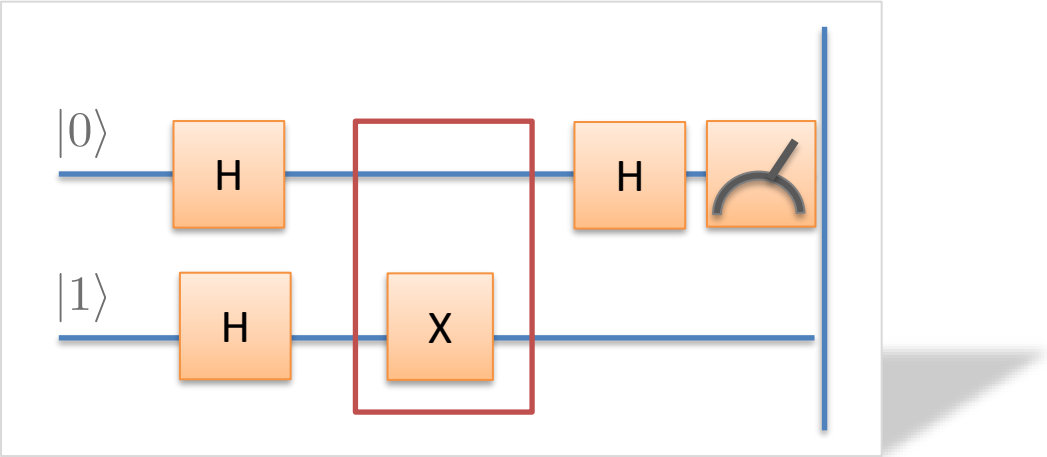
$$\mathrm{H}|+\rangle = \mathrm{H}\frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

$$= \frac{|0\rangle + |1\rangle + |0\rangle - |1\rangle}{2}$$

$$= |0\rangle$$

Hence: $\mathrm{H}|+\rangle = |0\rangle$

Deutsch algorithm: walkthrough

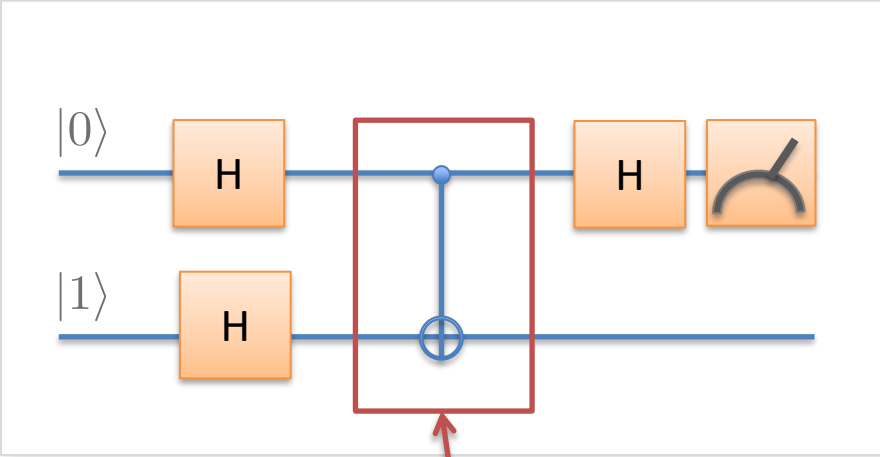


$$|\psi\rangle = -|0\rangle \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$
$$= -|0\rangle \otimes |-\rangle$$

We will measure “0” with 100% probability.
As we will see when we look at all cases, this indicates (with a single evaluation of the function) that the function is **constant**.

Deutsch algorithm: balanced function

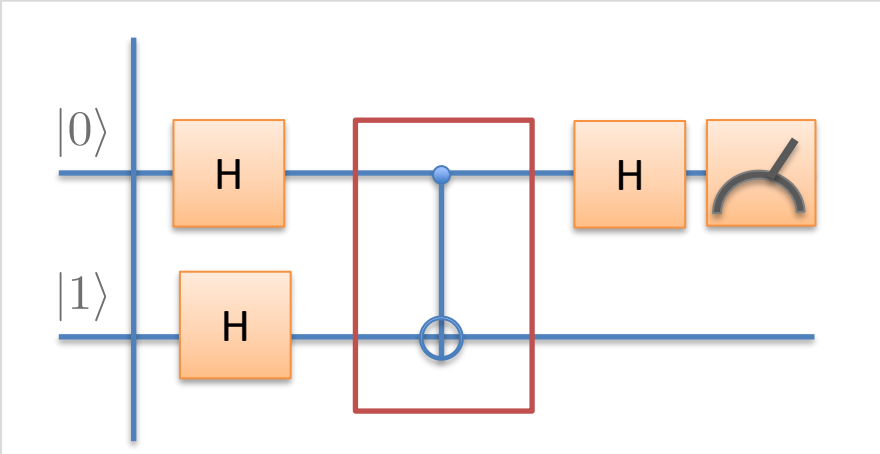
Now let's carefully look at the working of quantum circuit for the balanced case:



$$\begin{cases} f(0) = 0 \\ f(1) = 1 \end{cases}$$

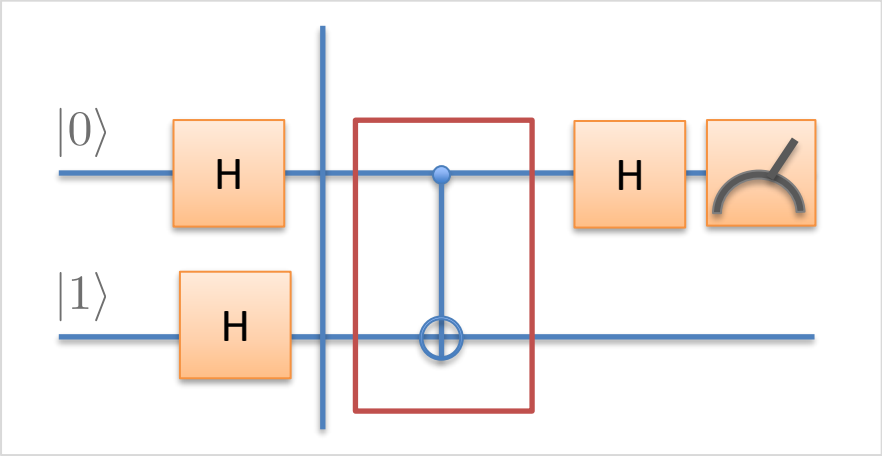
The function, U, is implemented by the gates inside this box

Deutsch algorithm: walkthrough



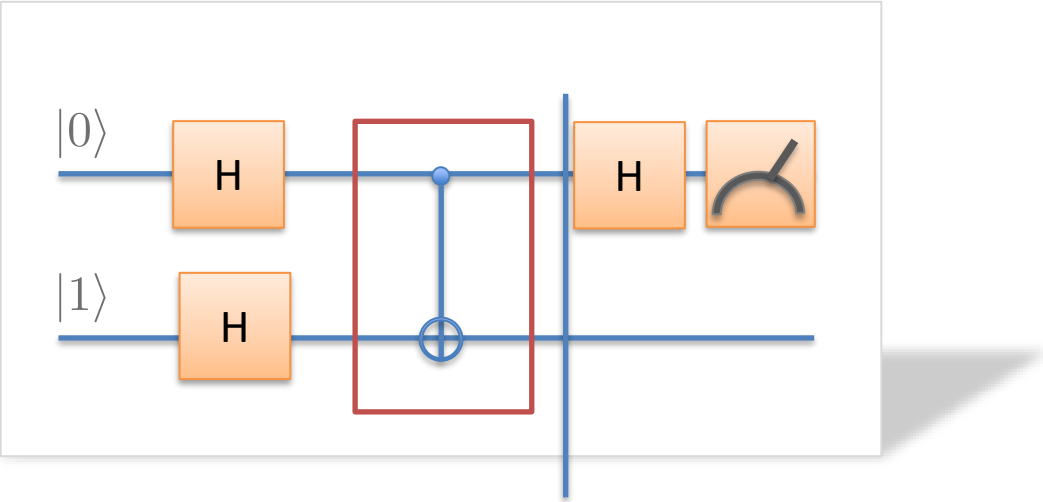
$$|\psi\rangle = |0\rangle \otimes |1\rangle$$

Deutsch algorithm: walkthrough



$$
\begin{aligned}
|\psi\rangle &= H\,|0\rangle \otimes H\,|1\rangle \\
&= \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
&= |+\rangle \otimes |-\rangle
\end{aligned}
$$

Deutsch algorithm: walkthrough

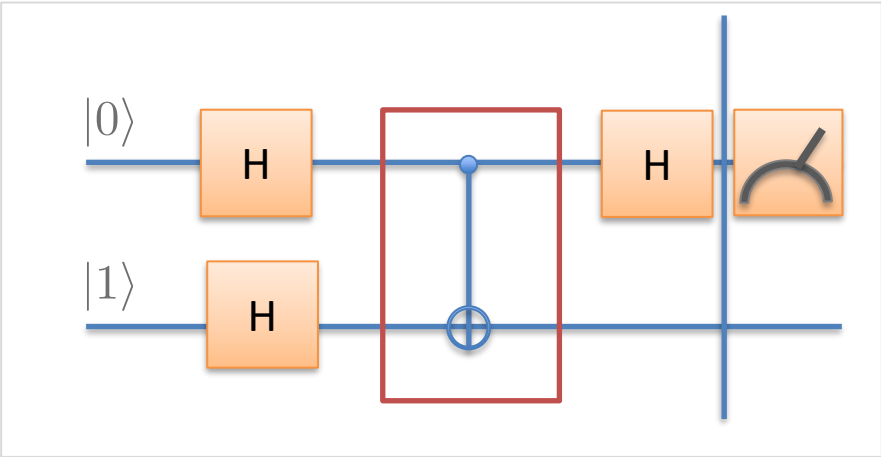


$$|\psi\rangle = \text{CNOT}|+\rangle|-\rangle = \text{CNOT}\frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

$$|\psi\rangle = \text{CNOT}\frac{|0\rangle|0\rangle - |0\rangle|1\rangle + |1\rangle|0\rangle - |1\rangle|1\rangle}{\sqrt{2}}$$

$$|\psi\rangle = \text{CNOT}\frac{|0\rangle|0\rangle - |0\rangle|1\rangle + |1\rangle|1\rangle - |1\rangle|0\rangle}{\sqrt{2}} = |-\rangle|-\rangle$$

Deutsch algorithm: walkthrough



$$|\psi\rangle = H\,|-\rangle \otimes |-\rangle$$
$$= |1\rangle \otimes |-\rangle$$

Note:

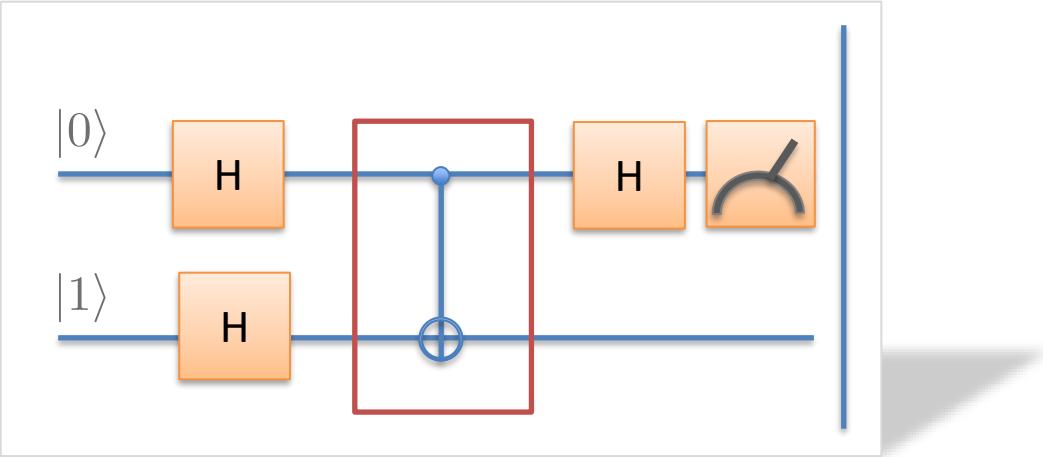
$$\mathrm{H}|-\rangle = \mathrm{H}\frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

$$= \frac{|0\rangle + |1\rangle - |0\rangle + |1\rangle}{2}$$

$$= |1\rangle$$

Hence: $\mathrm{H}|-\rangle = |1\rangle$

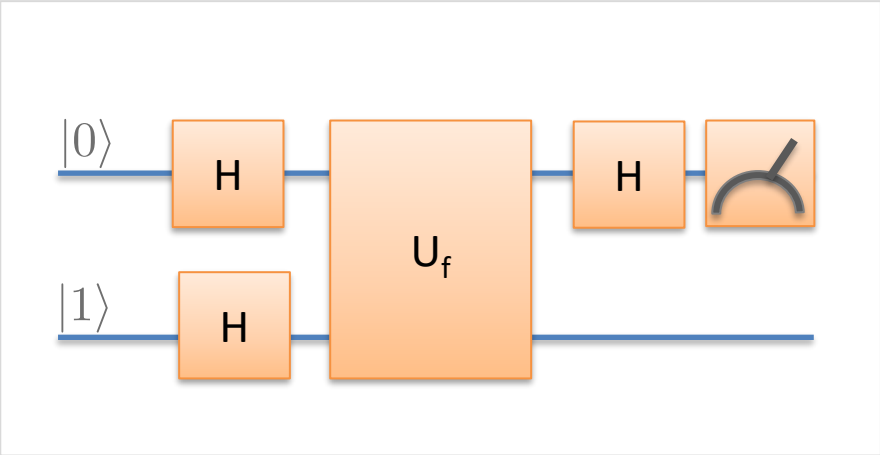
Deutsch algorithm: walkthrough



$$|\psi\rangle = |1\rangle \otimes |-\rangle$$

We will measure “1” with 100% probability. This indicates (with a single evaluation of the function) that the function is **balanced.**

Deutsch Algorithm-Summary



		Measured
CONSTANT	$f_1(0) = 0$ $f_1(1) = 0$	0
	$f_2(0) = 1$ $f_2(1) = 1$	0
BALANCED	$f_3(0) = 0$ $f_3(1) = 1$	1
	$f_4(0) = 1$ $f_4(1) = 0$	1

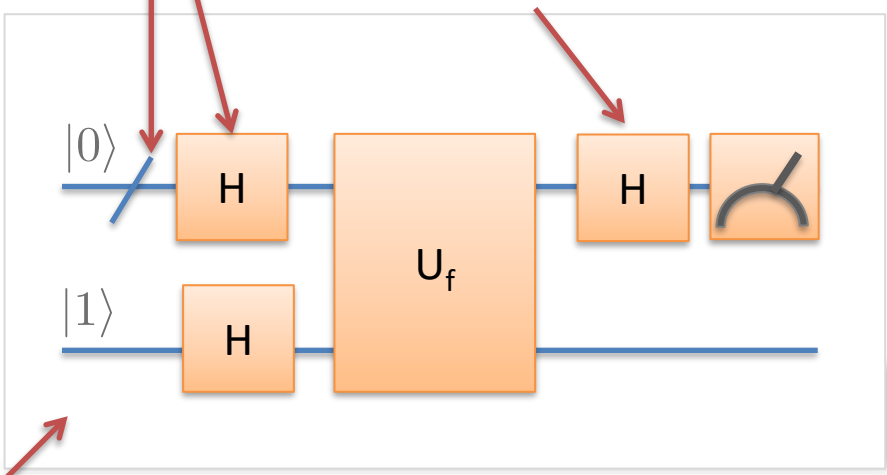
The Deutsch-Jozsa algorithm determines in just *one query* whether the function is constant or balanced.

Classically, this would require *two queries.*

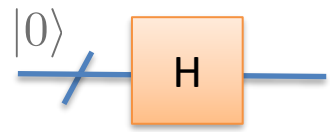
Multiple qubits Generalization: Deutsch-*Jozsa*

This means that there are multiple qubits in the register

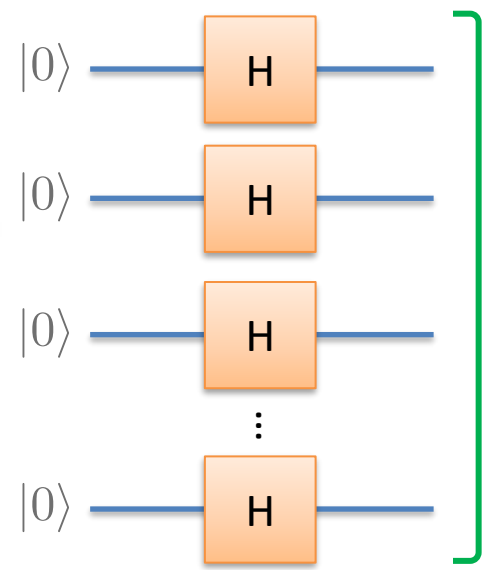
There are multiple Hadamard gates here



Only a single qubit here



n qubit register



The multi-qubit Deutsch-Josza algorithm will still require only one query to determine whether the function is constant or balanced.
Note: 0s in output register -> constant, any other valve -> balanced.

10.2 **Simon's Algorithm**

MULT20015

Lecture 10

Simon's Problem

Problem: Given a 2-to-1 function, f, such that

$$f(x) = f(x \oplus a)$$

Find a.

Unlike the previous two examples, here the range of f(x) is Z, integers.

Just like the previous algorithm, Simon's algorithm also did not have immediate real-world application. However, it is exponentially faster than the classical algorithm. It was the inspiration for Shor's factoring algorithm.

Example of a hidden a

f(001) = f(111)

x	f(x)
000	0
001	1
010	2
011	3
100	2
101	3
110	0
111	1

We would like to find the hidden 'a' s.t. $f(x) = f(x \oplus a)$

To find a, compute bitwise XOR between the two values of x that generate same f (x):

In this case: $a=110_2=6$ Why? $001 \oplus 111 = 110$

Complexity with Classical Computing

An approach is to try different inputs until you see a collision, this method r equires you to evaluate the function with many values of input x until you observe the same f(x) for two different inputs: $f(x_1) = f(x_2)$;

In a brute force approach, the number of inputs to be tried grows exponentially in general. i.e n =4 we have N = 16 possible inputs, in general $N = 2^n$.

We may note that Elementary Operation here is that evaluation of f (x) for a given value of x.
So how many elementary operations are actually required to find a?

f(000) = 0
f(011) = 3
f(111) = 1
f(010) = 2
f(001) = 1

Actually, this is equivalent to the famous "birthday" problem and takes fewer queries than you might expect. Probabilistically, if there are N = 2^n different inputs we need

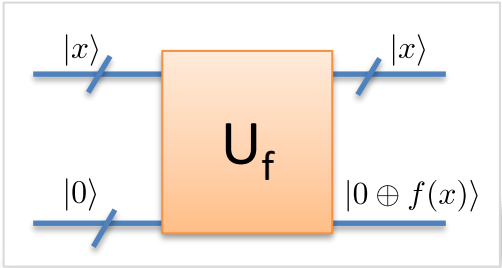
$$O(\sqrt{N}) = O\left(2^{\frac{n}{2}}\right) = O(2^n)$$

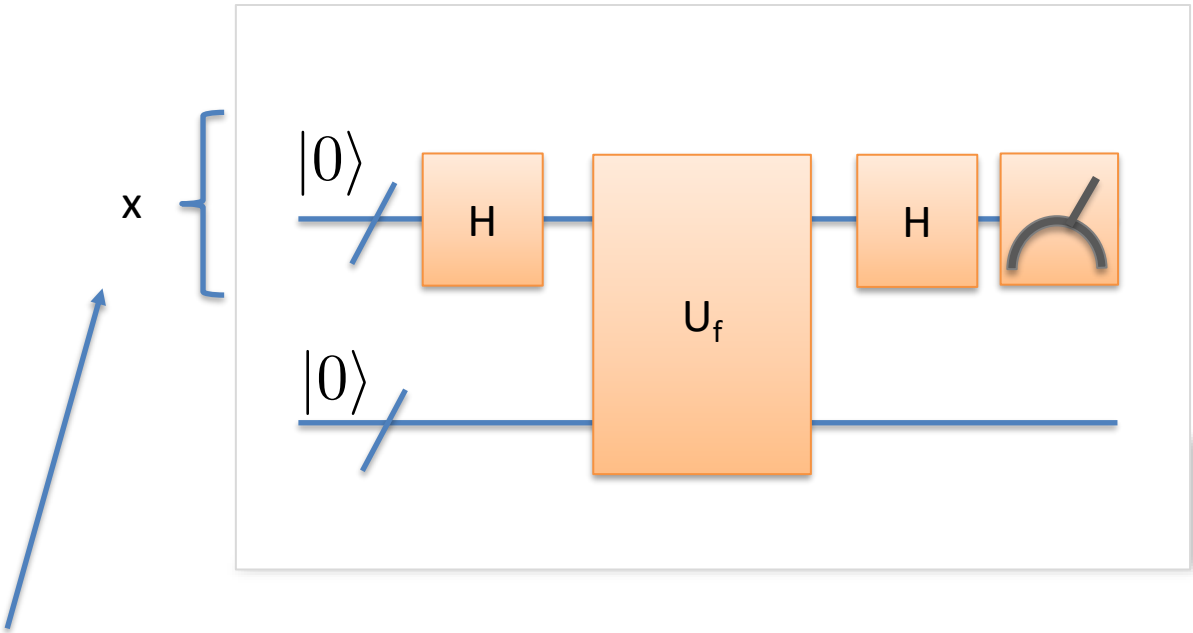
Evaluations f the function before we find a collision.

Simon's algorithm does the same with O(n) queries.

Simon's algorithm circuit

Randomly measure a result of the function.
Collapse to a superposition of inputs which give that value.
Send these through Hadamard gates, and measure:



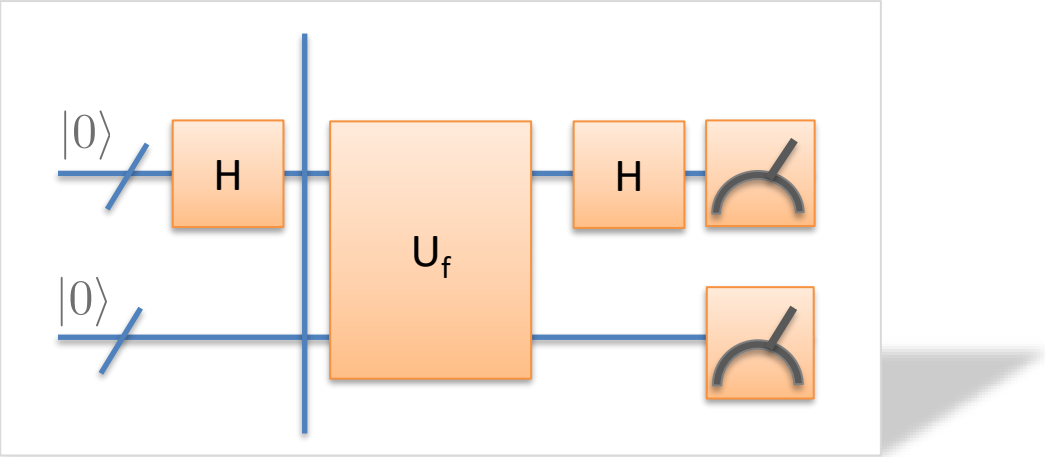


Measure to find a

$$f(x_0), f(x_0 \oplus a)$$

Note: *x* is a multi-qubit state [recall Lecture 8].

Simon's algorithm



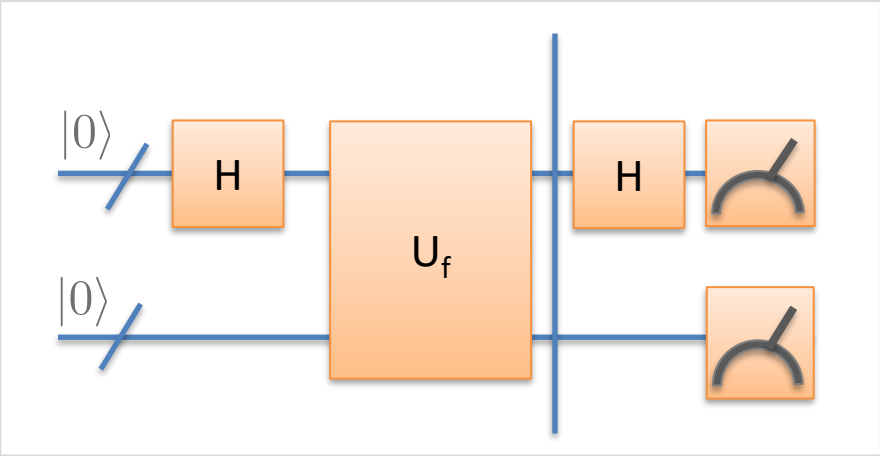
After the initial Hadamard gates:

Recall Lecture 8
Hadamard on n-qbit State

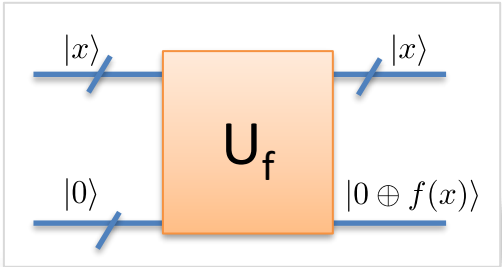
$$|\psi\rangle = \frac{1}{\sqrt{N}} \sum_x |x\rangle |0\rangle$$

Digital O State

Simon's algorithm



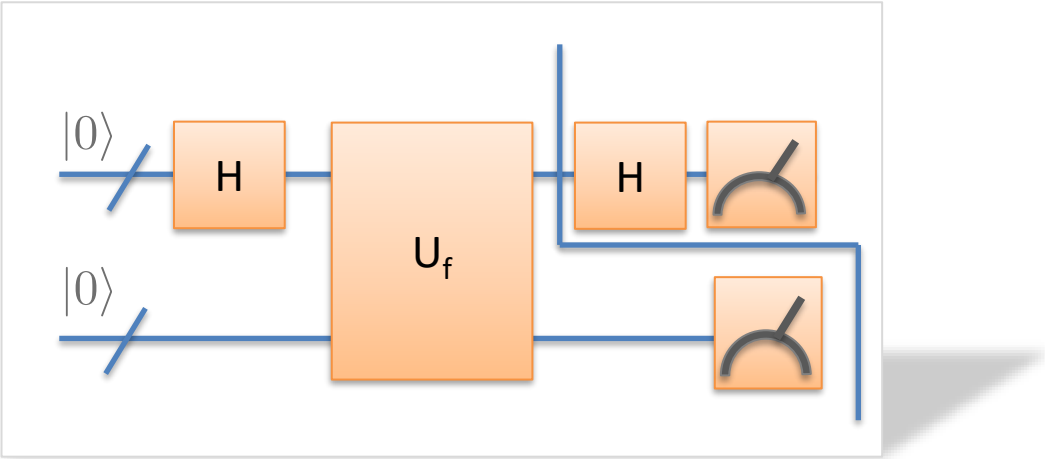
After evaluation of the function:



$$|\psi\rangle = \frac{1}{\sqrt{N}} \sum_x U_f \, |x\rangle \, |0\rangle$$

$$= \frac{1}{\sqrt{N}} \sum_x |x\rangle \, |f(x)\rangle$$

Simon's algorithm



It's easiest to consider that the bottom register is measured first. Before measurement the state is:

$$|\psi\rangle = \frac{1}{\sqrt{N}} \sum_x |x\rangle |f(x)\rangle$$

Some value, $f(x_0)$ will be measured at random, and the top register collapses to:

$$|\psi\rangle = \frac{|x_0\rangle + |x_0 \oplus a\rangle}{\sqrt{2}}$$

Example: Measuring function

$$|\psi\rangle = \frac{1}{\sqrt{N}} \sum_x |x\rangle |f(x)\rangle$$

$$= \frac{1}{\sqrt{8}} (|0\rangle |0\rangle + |1\rangle |1\rangle + |2\rangle |2\rangle + \boxed{|3\rangle |3\rangle} + |4\rangle |2\rangle + \boxed{|5\rangle |3\rangle} + |6\rangle |0\rangle + |7\rangle |1\rangle)$$

x	f(x)
000	0
001	1
010	2
011	3
100	2
101	3
110	0
111	1

If we measure the second register, and measure obtain "3", the state collapses to only those states compatible with this measurement:

$$|\psi'\rangle = \frac{|3\rangle |3\rangle + |5\rangle |3\rangle}{\sqrt{2}}$$

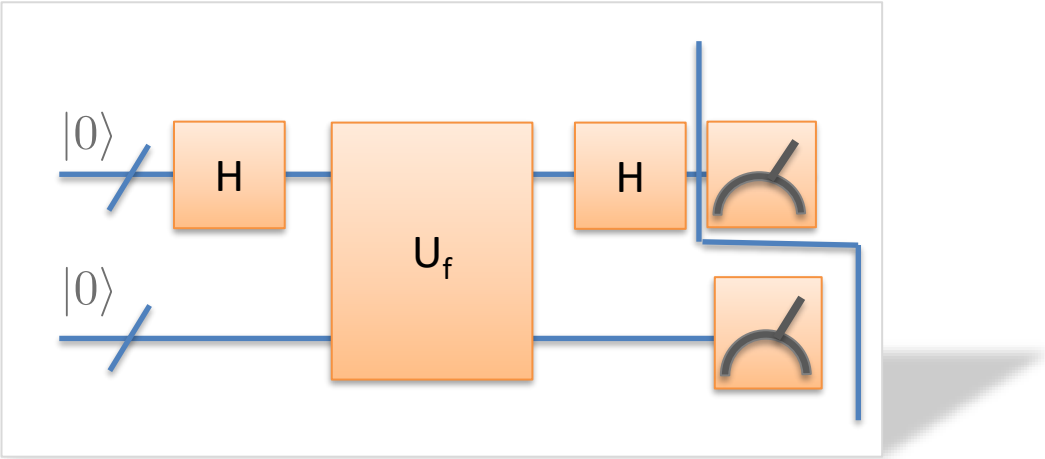
$$= \frac{|3\rangle + |5\rangle}{\sqrt{2}} \otimes |3\rangle$$

First register: $$|\psi\rangle = \frac{|x_0\rangle + |x_0 \oplus a\rangle}{\sqrt{2}}$$

Here x_0 is the value of x corresponding to the measured $f(x_0)$

Simon's algorithm

After measuring the bottom function register:



We now apply Hadamard to the top register (over all qubits):

$$|\psi\rangle = H^{\otimes n} \frac{|x_0\rangle + |x_0 \oplus a\rangle}{\sqrt{2}}$$

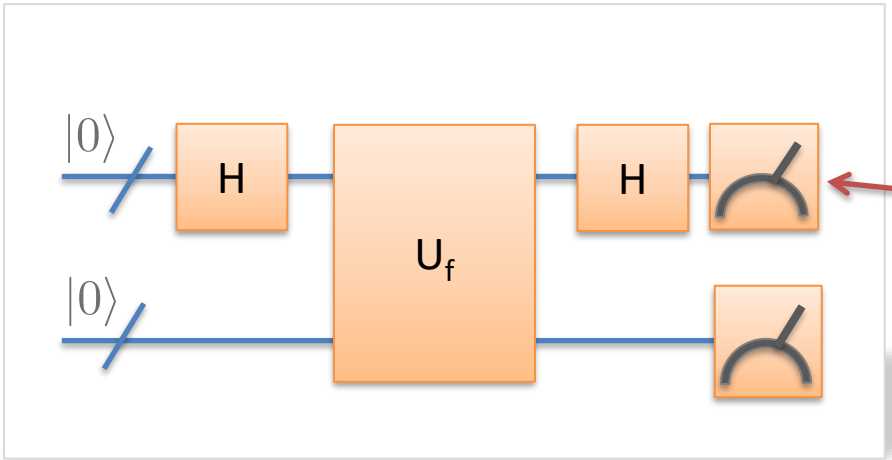
y is the measured value in the top register after H operation

Simon's algorithm

Constraint on the measured value of *y*

After H gate on first/upper output, you will only measure *y*'s for which:

$$a \cdot y = 0 \mod 2$$



$$a \cdot y = 0 \mod 2$$

Note:

$a \,.\, y = a_1y_1 + a_2y_2 + a_3y_3$

Each time we measure, we randomly measure a "y" which is orthogonal to "a":

Example of Simon's algorithm

x	f(x)
000	0
001	1
010	2
011	3
100	2
101	3
110	0
111	1

We would like to find the hidden 'a' s.t.

$$f(x) = f(x \oplus a)$$

In this case, a=110_2=6

We run the circuit, and at random, measure the results:

$$a \cdot y = 0 \mod 2$$

$\longrightarrow$ *y* =

001
110
111

Finding a

Now, how do we find a?

We know that $$a \cdot y = 0 \mod 2$$

We have three values of 'y' for which this is true, so we can write a system of linear equations for the bits of 'a':

Measured values

001
110
111

$$\left.\begin{aligned} a_3 &= 0 \\ a_1 + a_2 &= 0 \\ a_1 + a_2 + a_3 &= 0 \end{aligned}\right\}$$

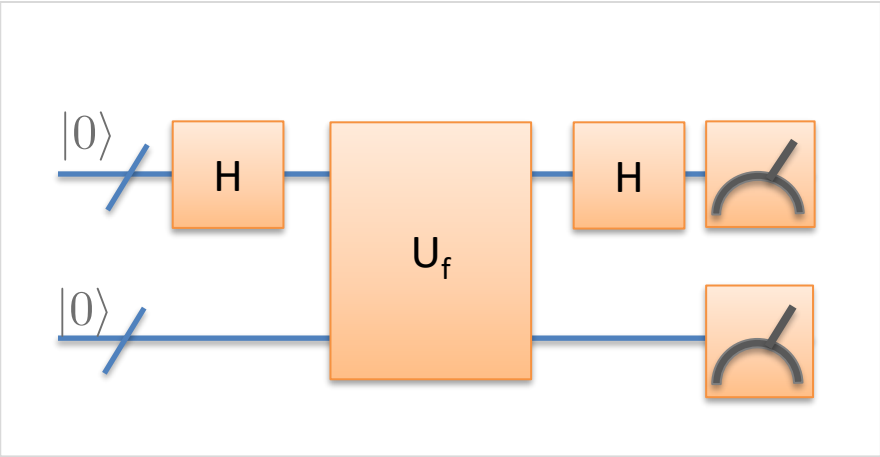
Note:

a is a 3-bit number: (a_1,a_2,a_3)

Solving for a_1, a_2, and a_3

Solution is degenerate. a =(0,0,0) or a_1= a_2=1 i.e. a=(1,1,0)

Simon's Algorithm: Summary



Given a 2-to-1 function, f, such that

$$f(x) = f(x \oplus a)$$

Find a.

Classical algorithm: $O(\sqrt{N}) = O\left(2^{\frac{n}{2}}\right) = O(2^n)$ Queries to the oracle (probabilistically)

Quantum algorithm: $O(n)$ Queries to the oracle

Subject outline

Lecture topics (by week)

1 – Introduction to quantum computing and maths basics
2 – Single qubit representations and logic operations
3 – Two qubit states and logic gates
4 – Multi-qubit states and quantum arithmetic
5 – Basic quantum algorithms
6 – Period finding, cryptography and quantum factoring
7 – Shor's algorithm, post-quantum crypto, quantum key distribution
8 – Quantum search algorithms
9 – Grover search applications, optimisation problems
10 – Solving optimisation problems on quantum computers
11 – Applications in quantum machine learning
12 – Real quantum computer devices

Assignment schedule:
#1: Hand out in Week 2
#2: Hand out in Week 8